

Regularization and Evaluation

Hrachya Asatryan

Regularization

Main Principle:

To control over-fitting, a regularization term to the error function can be introduced in the form of:

$$E_D(\mathbf{w}) + \lambda E_W(\mathbf{w})$$

where λ is the regularization coefficient that controls the relative importance of the data-dependent error $E_D(w)$ and the regularization term $E_W(w)$. One of the simplest forms of regularizer is given as:

$$E_W(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

If we also consider the sum-of-squares error function given by:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2$$

then the total error function becomes:

$$\frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2 + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}.$$

Regularization

Main Principle:

This choice of regularizer is known as **weight decay**, as in sequential algorithms it encourages weight values to decay towards zero, unless supported by data. In statistics, it shrinks parameter values towards zero. It has the advantage that the error function remains a quadratic function of $\mathbf{w}$, and so its exact minimizer can be found in closed form. Specifically, setting the gradient of with respect to $\mathbf{w}$ to zero, and solving for $\mathbf{w}$ as before, we obtain:

$$\mathbf{w} = (\lambda \mathbf{I} + \Phi^T \Phi)^{-1} \Phi^T \mathbf{t}$$

This represents a simple extension of the least-squares solution.

A more general regularizer is sometimes used, for which the regularized error takes the form of:

$$\frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2 + \frac{\lambda}{2} \sum_{j=1}^M |w_j|^q$$

Where $q = 2$ corresponds to the quadratic regularizer. The case of $q = 1$ is known as the **lasso**, if λ is sufficiently large, some of the coefficients w_j are driven to zero, leading to a sparse model in which the corresponding basis functions play no role.

Regularization

Why Does Linear Regression Need Regularization?

Ordinary Least Squares minimizes

$$\min_w ||y - Xw||^2$$

However, this approach can fail in several situations:

- **High dimensional data** (many features)
- **Strongly correlated predictors**
- **Small datasets**
- **Ill-conditioned design matrices**

Consequences:

- Extremely large coefficients
- Highly unstable models
- Poor generalization

Regularization stabilizes the solution by **penalizing model complexity**.

Illustration

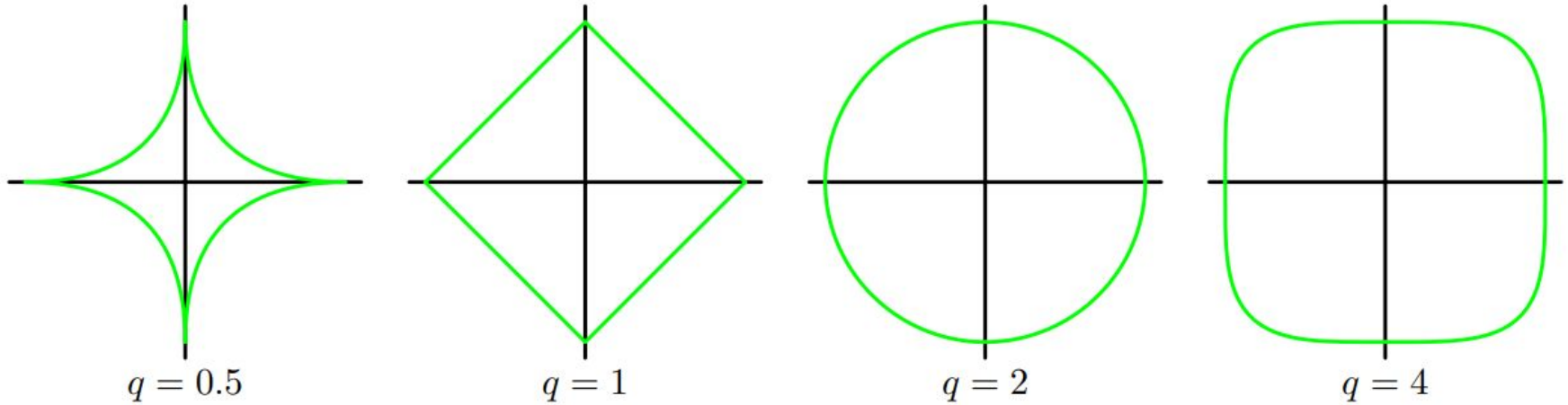
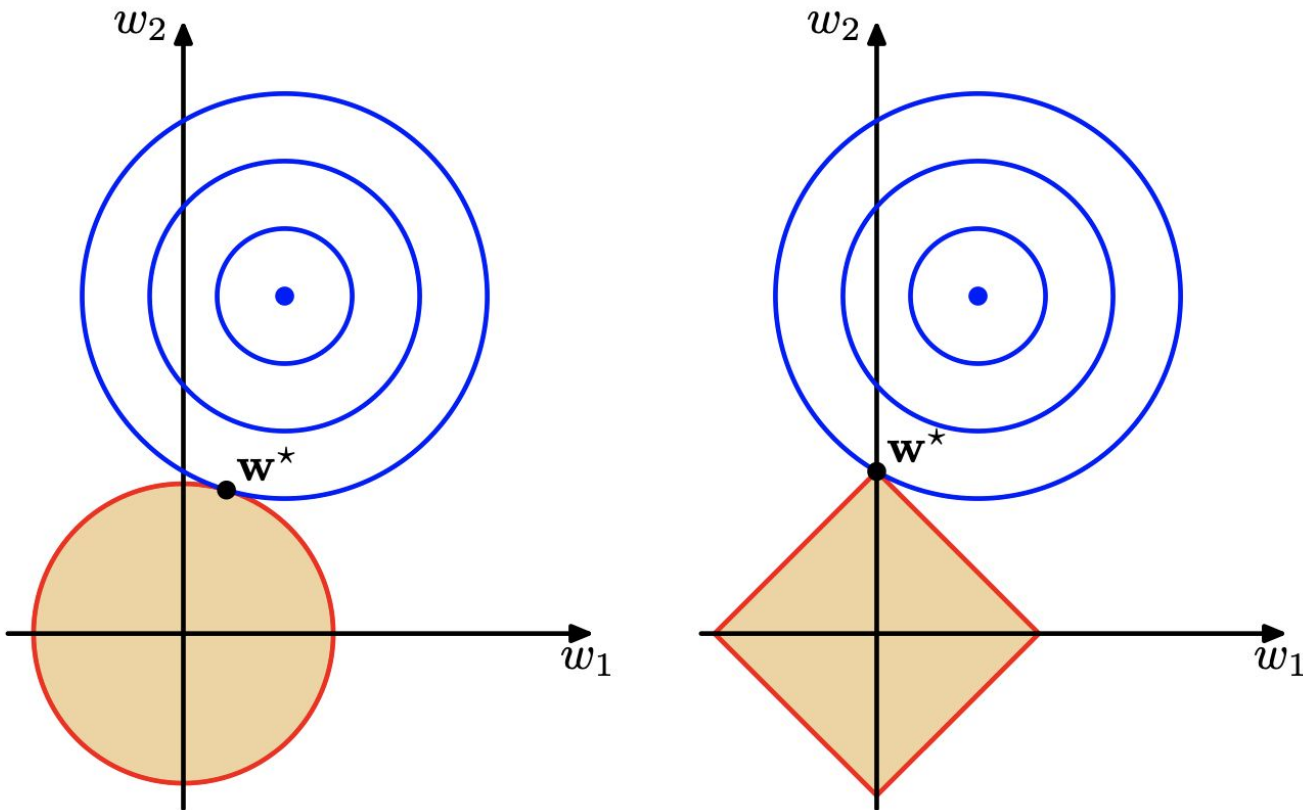


Figure 3.3 Contours of the regularization term in (3.29) for various values of the parameter q .

Illustration

Plot of the contours of the unregularized error function (blue) along with the constraint region for the quadratic regularizer. $q = 2$ on the left and $q = 1$ on the right.

The optimum value for the parameter vector w is denoted by w^* . The lasso gives a sparse solution in which $w_1^* = 0$.



Ridge Regression

Main Principle:

Ridge regression shrinks the regression coefficients by imposing a penalty on their size. The ridge coefficients minimize a penalized residual sum of squares.

$$\hat{\beta}^{\text{ridge}} = \underset{\beta}{\operatorname{argmin}} \left\{ \sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}$$

Here $\lambda \geq 0$ is a complexity parameter that controls the amount of shrinkage: the larger the value of λ , the greater the amount of shrinkage. The coefficients are shrunk toward zero (and each other).

We can also write this as:

$$\begin{aligned} \hat{\beta}^{\text{ridge}} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 \\ \text{subject to } \sum_{j=1}^p \beta_j^2 \leq t, \end{aligned}$$

Making the explicit the size constraint on the parameters more explicit.

Ridge Regression

Main Principle:

In a linear regression model with many correlated variables, the coefficients can become unstable, with large positive and negative values canceling each other out. Imposing a size constraint on the coefficients helps fix this issue. The ridge solutions are not equivariant under scaling of the inputs, and so one normally standardizes the inputs before solving.

The intercept β_0 has been left out of the penalty term, as penalizing the intercept would make the procedure depend on the origin chosen for Y , that is, adding a constant c to each of the targets y_i would not simply result in a shift of the predictions by the same amount c .

It can be shown that the solution to can be separated into two parts, after re-parameterization using centered inputs: each x_{ij} gets replaced by $x_{ij} - x_j$. We estimate β_0 by:

$$\bar{y} = \frac{1}{N} \sum_1^N y_i$$

Ridge Regression

Main Principle:

The remaining coefficients are estimated using ridge regression without the intercept, based on centered data x_{ij} . We assume the centering is done, so the input matrix $\mathbf{X}$ has p columns instead of $p + 1$.

Writing the criterion in matrix form, we get:

$$\text{RSS}(\lambda) = (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) + \lambda \beta^T \beta,$$

The ridge regression solutions are easily seen to be:

$$\hat{\beta}^{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y},$$

where $\mathbf{I}$ is the $p \times p$ identity matrix.

In the case of orthonormal inputs, the ridge estimates are just a scaled version of the least squares estimates, that is:

$$\hat{\beta}^{\text{ridge}} = \hat{\beta} / (1 + \lambda)$$

Ridge Regression

Ridge Regression as a Numerical Stabilizer

The OLS solution is:

$$\hat{w} = (X^T X)^{-1} X^T y$$

If $X^T X$ is **ill-conditioned or nearly singular**, inversion becomes unstable.

Ridge regression modifies the solution:

$$\hat{w}_{ridge} = (X^T X + \lambda I)^{-1} X^T y$$

Key property:

Adding λI **shifts eigenvalues away from zero**. If $X^T X = Q \Lambda Q^T$, then $X^T X + \lambda I = Q(\Lambda + \lambda I)Q^T$

Thus eigenvalues become:

$$\lambda_i + \lambda$$

This guarantees a **stable inverse**.

Lasso Regression

Main Principle:

The lasso is a shrinkage method like ridge, with subtle but important differences. The lasso estimate is defined by:

$$\hat{\beta}^{\text{lasso}} = \underset{\beta}{\operatorname{argmin}} \left\{ \frac{1}{2} \sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j| \right\}$$

Just as in ridge regression, we can re-parametrize the constant β_0 by standardizing the predictors; the solution for $\hat{\beta}_0$ is $\bar{y}$, and thereafter we fit a model without an intercept. We can also write the lasso problem in the equivalent Lagrangian form:

$$\begin{aligned} \hat{\beta}^{\text{lasso}} &= \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 \\ &\text{subject to } \sum_{j=1}^p |\beta_j| \leq t. \end{aligned}$$

Lasso Regression

Main Principle:

Notice the similarity to the ridge regression problem: the **L2 ridge penalty** is replaced by the **L1 lasso penalty**. This latter constraint makes the solutions nonlinear in the y_i , and there is no closed form expression as in ridge regression. Computing the lasso solution is a quadratic programming problem, however, efficient algorithms are available for computing the entire path of solutions as λ is varied, with the same computational cost as for ridge regression.

Making t sufficiently small will cause some coefficients to be exactly 0, thus the lasso does a kind of continuous subset selection. Like the **penalty parameter** in ridge regression, t should be adaptively chosen to minimize an estimate of expected prediction error.

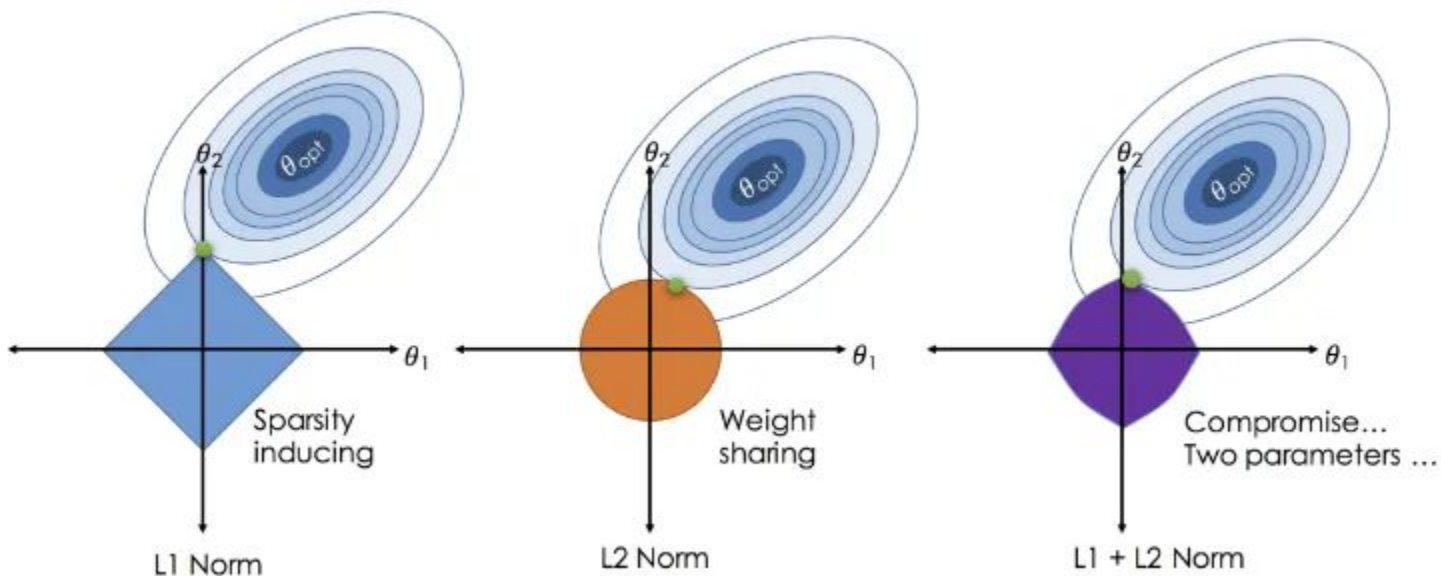
ElasticNet

$$\text{Elastic Net Loss} = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left(\alpha \sum_{j=1}^p |\beta_j| + (1 - \alpha) \sum_{j=1}^p \beta_j^2 \right)$$

Here:

1. $\frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$: Mean squared error (MSE), the regression loss term measuring the difference between predicted and actual values.
2. λ : Regularization parameter, controlling the overall strength of the regularization.
3. α : Mixing parameter, determining the balance between L1 (Lasso) and L2 (Ridge) penalties.
4. $\sum_{j=1}^p |\beta_j|$: L1 penalty, inducing sparsity.
5. $\sum_{j=1}^p \beta_j^2$: L2 penalty, encouraging small coefficients.

ElasticNet



Comparison

This section compares three ways to limit a linear regression model: subset selection, ridge regression, and the lasso. When the input matrix X is orthonormal, each method has a clear solution, applying a simple transformation to the least squares estimate:

- **Ridge regression** shrinks all coefficients proportionally.
- **Lasso** reduces each coefficient by a constant factor λ , truncating at zero (called soft thresholding), setting small ones to zero.
- **Subset selection** drops all variables with coefficients smaller than the M th largest (hard thresholding), dropping the rest.

For non-orthogonal data, visualizations help. The illustration on the next slide, depicts **the lasso** (left) and **ridge regression** (right) when there are only two parameters. The residual sum of squares has elliptical contours, centered at the full least squares estimate. The constraint region for ridge regression is the disk $\beta_1^2 + \beta_2^2 \leq t$, while that for lasso is the diamond $|\beta_1| + |\beta_2| \leq t$. Both methods find the first point where the elliptical contours hit the constraint region, however in the case of the lasso, as diamond has corners; if the solution occurs at a corner, then it has one parameter β_j equal to zero. When $p > 2$, the diamond becomes a rhomboid, and has many corners, flat edges and faces; there are many more opportunities for the estimated parameters to be zero.

Illustration

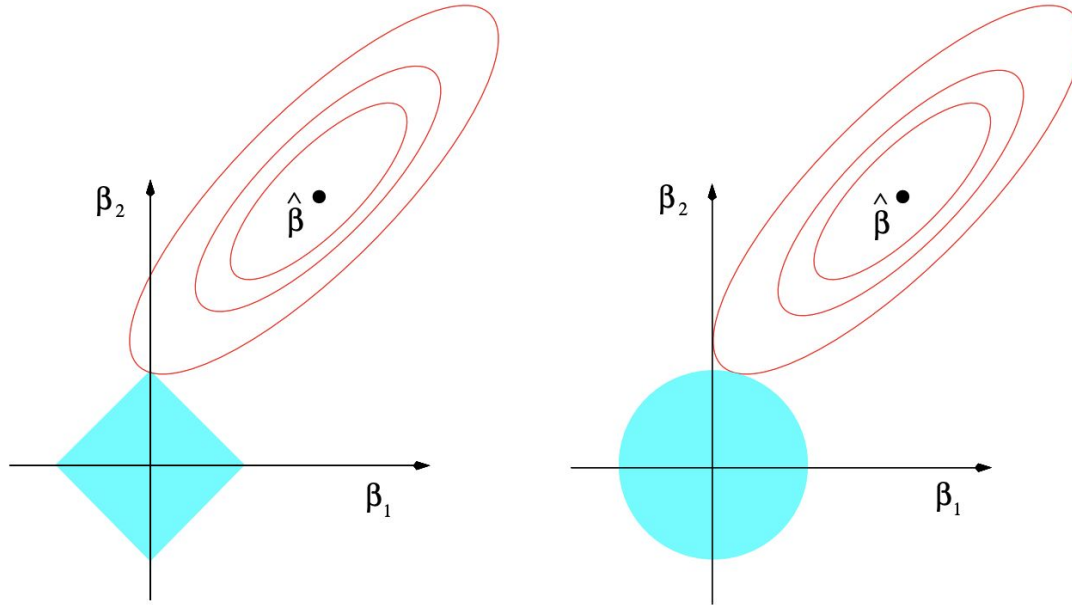


FIGURE 3.11. Estimation picture for the lasso (left) and ridge regression (right). Shown are contours of the error and constraint functions. The solid blue areas are the constraint regions $|\beta_1| + |\beta_2| \leq t$ and $\beta_1^2 + \beta_2^2 \leq t^2$, respectively, while the red ellipses are the contours of the least squares error function.

Geometric Interpretation

Regularization can also be written as a **constrained optimization problem**.

Ridge:

$$\min_w ||y - Xw||^2$$

subject to

$$||w||_2^2 \leq t$$

Lasso:

$$\min_w ||y - Xw||^2$$

subject to

$$||w||_1 \leq t$$

The solution occurs where the **error contours intersect the constraint region**.

Because the L1 constraint has **corners**, solutions often lie exactly on axes → coefficients become **zero**.

Comparison

Lasso Regression	Ridge Regression
❑ Uses L1 regularization - absolute value of coefficients, ❑ Can force the coefficients to be exactly 0, ❑ Performs both regularization and feature selection, making it more suitable for high dimensional data, ❑ May be more suitable in situations where only a subset of features contribute significantly to the output, ❑ Can lead to a sparse model - the model will have fewer features.	❑ Uses L2 regularization - square of coefficients, ❑ Shrinks the coefficients of less significant features towards 0, ❑ Does not performs feature selection, making and can only shrink coefficients values. This makes it more suitable for datasets with highly correlated predictors, since it avoids including all of them in the model, ❑ Works better when there are fewer significant features, ❑ Does not produce sparse models.

Correlated Features Behavior

When predictors are highly correlated:

OLS:

- Coefficients become unstable
- Large positive and negative values cancel each other
- Small changes in data $\rightarrow$ large coefficient changes

Ridge Regression:

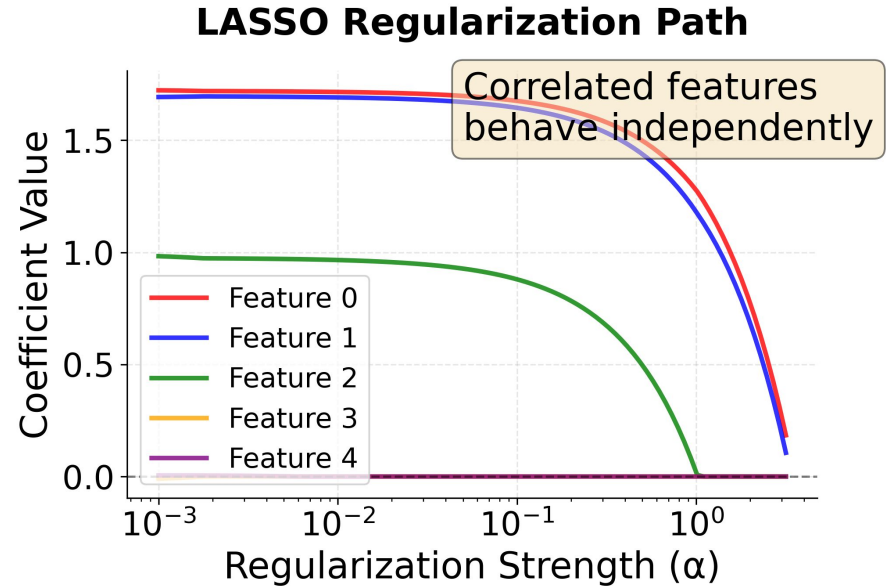
- Distributes weight across correlated features
- Produces stable solutions

Lasso:

- Tends to **select one feature and suppress others**

Elastic Net:

- Combines both effects
- Useful when many correlated predictors exist



How do we choose λ ?

The regularization strength λ controls the tradeoff:

Small λ :

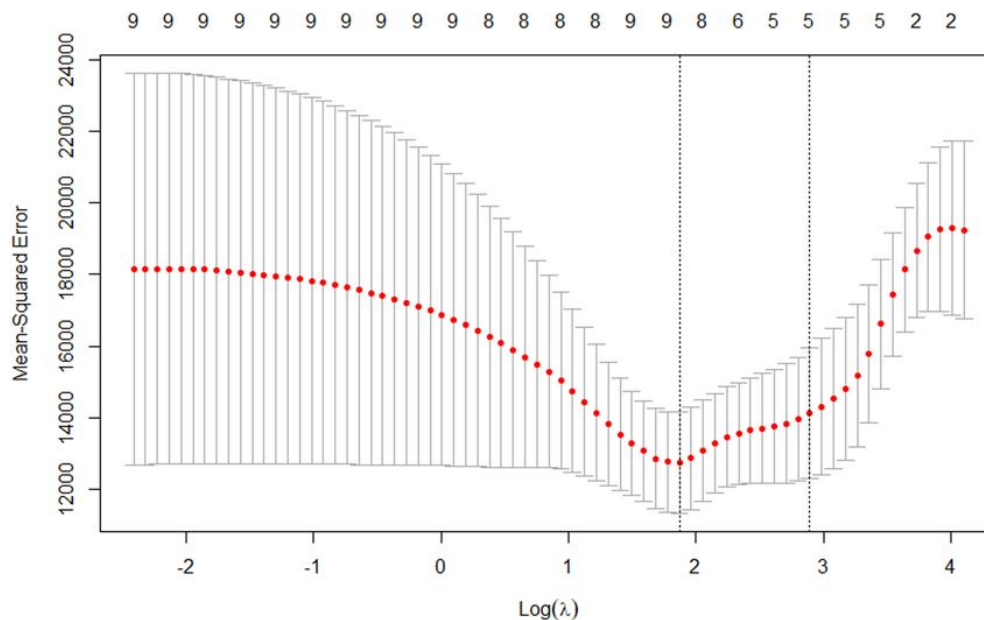
- Model close to OLS
- More prone to overfitting

Large λ :

- Strong shrinkage
- More prone to underfitting

In practice, λ is chosen using **cross-validation**:

1. Train models for many λ values
2. Evaluate validation error
3. Choose λ minimizing validation error



Bias-Variance

The Bias-Variance Decomposition:

In case of fitting **Regression Models**, a popular choice is the **squared loss function**, for which the optimal prediction is given by the conditional expectation, which we denote by $h(x)$ and which is given by:

$$h(\mathbf{x}) = \mathbb{E}[t|\mathbf{x}] = \int tp(t|\mathbf{x}) dt$$

The expected squared loss can be written in the form:

$$\mathbb{E}(L) = \int \int L(t, y(\mathbf{x}))p(\mathbf{x}, t)d\mathbf{x}dt = \int \int [y(\mathbf{x}) - t]^2 p(\mathbf{x}, t)d\mathbf{x}dt$$

Now modifying this we get:

$$\begin{aligned}(y(\mathbf{x}) - t)^2 &= (y(\mathbf{x}) - h(\mathbf{x}) + h(\mathbf{x}) - t)^2 \\ &= (y(\mathbf{x}) - h(\mathbf{x}))^2 + 2(y(\mathbf{x}) - h(\mathbf{x}))(h(\mathbf{x}) - t) + (h(\mathbf{x}) - t)^2\end{aligned}$$

Decomposition

The Bias-Variance Decomposition:

Taking the expectation over both sides of the equation over $\mathbf{x}$ and t we get:

$$\mathbb{E}_{x,t}[y(\mathbf{x}) - t]^2 = \mathbb{E}_{x,t}[(y(\mathbf{x}) - h(\mathbf{x}))^2] + 2\mathbb{E}_{x,t}[(y(\mathbf{x}) - h(\mathbf{x}))(h(\mathbf{x}) - t)] + \mathbb{E}_{x,t}[(h(\mathbf{x}) - t)^2]$$

As $h(\mathbf{x}) = E[t|\mathbf{x}]$, it follows that $E_{x,t}[h(\mathbf{x}) - t] = 0$, thus the second term of the equation above is 0.

Thus we achieve the the formula of:

$$\mathbb{E}[L] = \int \{y(\mathbf{x}) - \mathbb{E}[t|\mathbf{x}]\}^2 p(\mathbf{x}) d\mathbf{x} + \int \{\mathbb{E}[t|\mathbf{x}] - t\}^2 p(\mathbf{x}) d\mathbf{x}$$

Our first term here is the **model error** and our second term is the **irreducible noise**. The second term, which is independent from $\mathbf{y}(\mathbf{x})$ arises from intrinsic noise on the data and represents the minimum achievable value of the expected loss. The first term depends on our choice for the function $\mathbf{y}(\mathbf{x})$, and we will seek a solution for $\mathbf{y}(\mathbf{x})$ which makes this term a minimum and as it's non-negative, its smallest value can be 0.

Decomposition

The Bias-Variance Decomposition:

Suppose we had a large number of data sets each of size N and each drawn independently from the distribution $p(t, \mathbf{x})$. For any given data set D , we can run our learning algorithm and obtain a prediction function $y(\mathbf{x}; D)$. Different data sets from the ensemble will give different functions and consequently different values of the squared loss. The performance of a particular learning algorithm is then assessed by taking the average over this ensemble of data sets.

The first term in the previous equation for a particular dataset D takes the form:

$$\{y(\mathbf{x}; D) - h(\mathbf{x})\}^2$$

Following the logic mentioned above, we will take the average over the ensemble of datasets. Adding and subtracting $\mathbb{E}_D(y(\mathbf{x}; D))$ in the equation above we get:

$$\begin{aligned} & \{y(\mathbf{x}; D) - \mathbb{E}_D[y(\mathbf{x}; D)] + \mathbb{E}_D[y(\mathbf{x}; D)] - h(\mathbf{x})\}^2 \\ &= \{y(\mathbf{x}; D) - \mathbb{E}_D[y(\mathbf{x}; D)]\}^2 + \{\mathbb{E}_D[y(\mathbf{x}; D)] - h(\mathbf{x})\}^2 \\ & \quad + 2\{y(\mathbf{x}; D) - \mathbb{E}_D[y(\mathbf{x}; D)]\}\{\mathbb{E}_D[y(\mathbf{x}; D)] - h(\mathbf{x})\}. \end{aligned}$$

Decomposition

The Bias-Variance Decomposition:

As before, taking the expectation over the both sides over D , we get:

$$\begin{aligned} \mathbb{E}_{\mathcal{D}} [\{y(\mathbf{x}; \mathcal{D}) - h(\mathbf{x})\}^2] \\ = \underbrace{\{\mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})] - h(\mathbf{x})\}^2}_{(\text{bias})^2} + \underbrace{\mathbb{E}_{\mathcal{D}} [\{y(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})]\}^2]}_{\text{variance}} \end{aligned}$$

The first term, - **bias**², represents the extent to which the average prediction over all datasets differs from the desired regression function.

The second term - **variance**, measures the extent to which the solutions for individual data sets vary around their average, and hence this measures the extent to which the function $y(\mathbf{x}; D)$ is sensitive to the particular choice of data set.

Trade-off

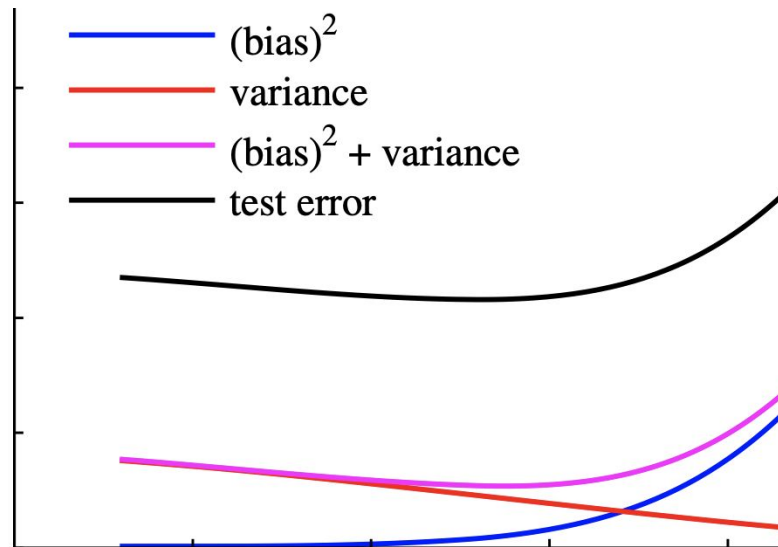
The Bias-Variance Trade-off:

Finally substituting this back into the equation in page 49, we get: **expected loss=(bias)²+variance+noise**

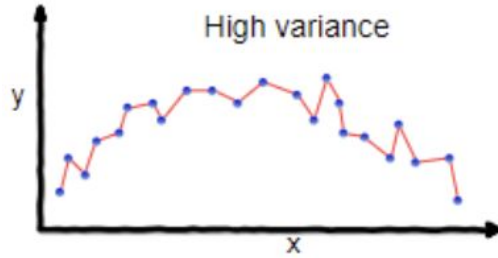
$$(\text{bias})^2 = \int \{\mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})] - h(\mathbf{x})\}^2 p(\mathbf{x}) d\mathbf{x}$$

$$\text{variance} = \int \mathbb{E}_{\mathcal{D}} [\{y(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})]\}^2] p(\mathbf{x}) d\mathbf{x}$$

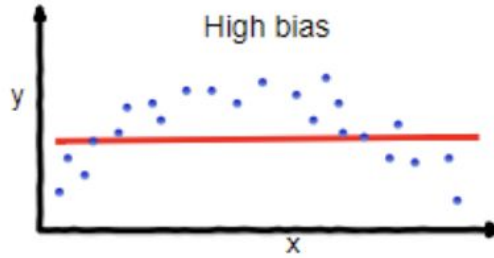
$$\text{noise} = \int \{h(\mathbf{x}) - t\}^2 p(\mathbf{x}, t) d\mathbf{x} dt$$



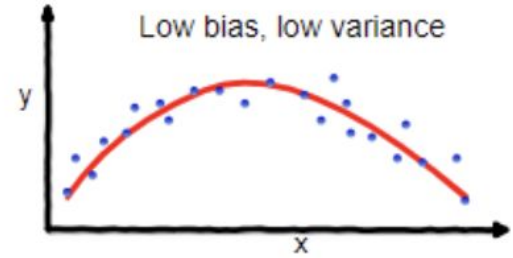
Trade-off



overfitting



underfitting

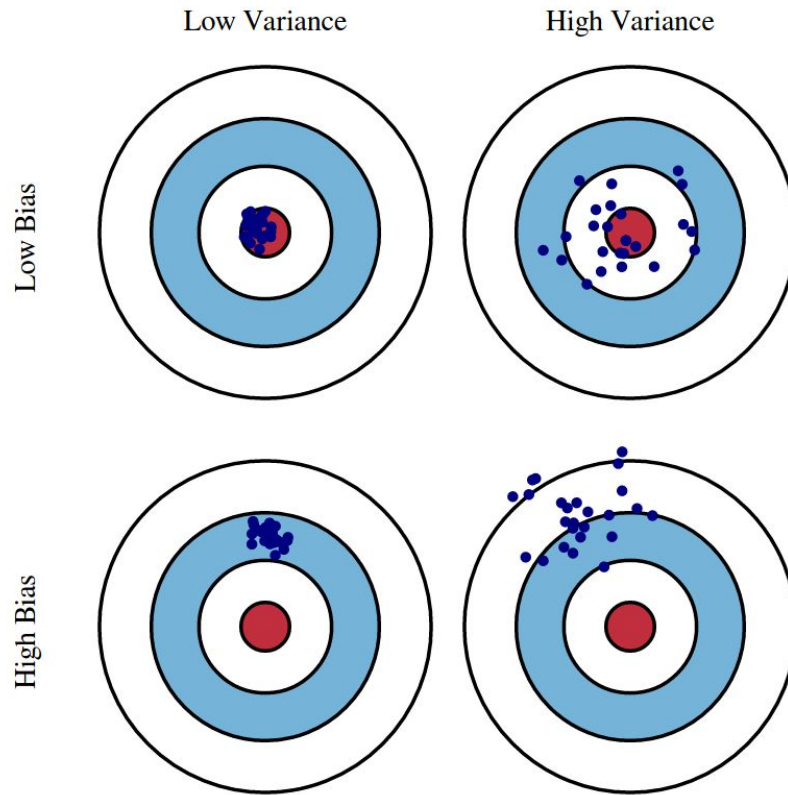


Good balance

Figure taken from:

<https://towardsdatascience.com/understanding-the-bias-variance-tradeoff-165e6942b229>

Illustration



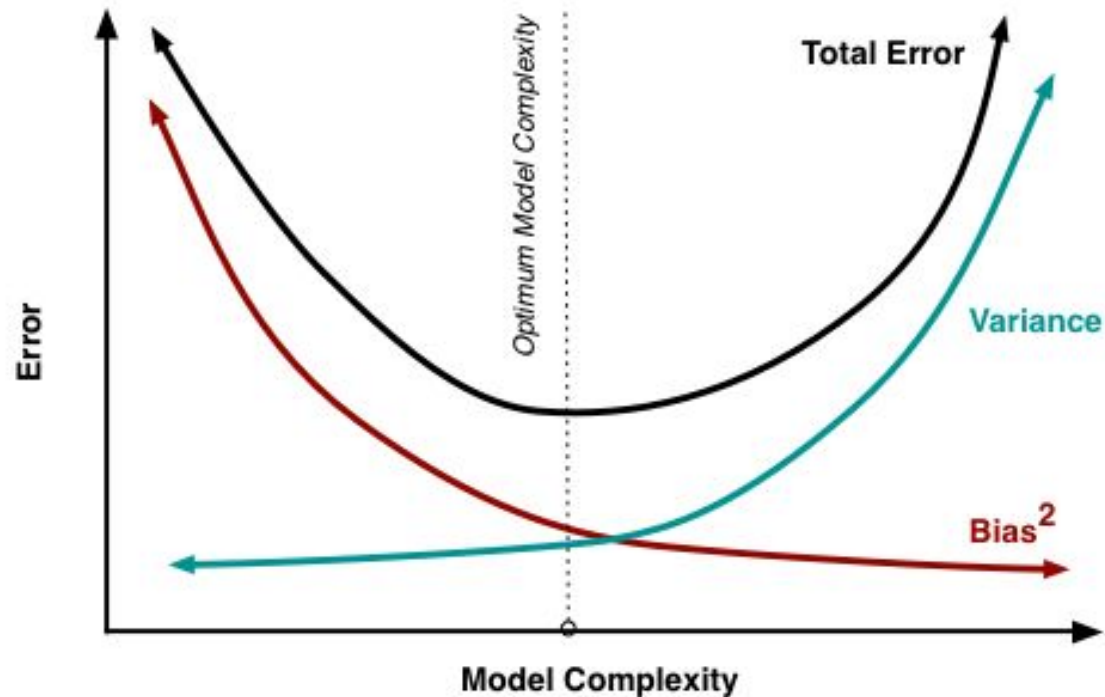
Trade-off

The Bias-Variance Trade-off:

As our goal is to minimize our expected loss, we will see that there is a trade-off between bias and variance:

- Very flexible models have **low bias and high variance**
- Relatively rigid models have **high bias and low variance**

The model with the optimal predictive capability is the one that leads to the best balance between bias and variance.



From Model Complexity to Model Evaluation

Why do we need evaluation metrics?

We have seen that increasing model complexity affects:

- Bias
- Variance
- Generalization ability

But in practice we still need to answer:

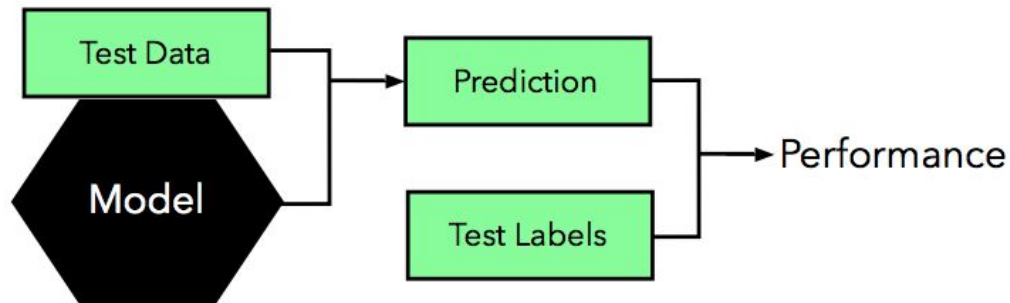
How do we measure whether a model is good?

Different tasks require different metrics.

Regression → continuous outputs

Classification → discrete labels

We now introduce common evaluation metrics used in machine learning.



Evaluation metrics

Why do we need evaluation metrics?

Let's say we have trained a few models, that perform well on the train data. How can we compare said models? What sort of metrics can we use to choose the best model out of them, or to see if our model is even viable in the "real world"? For regression models, the most commonly used metrics are:

- Mean Absolute Error (**MAE**)
- Mean Squared Error (**MSE**)
- Root Mean Squared Error (**RMSE**)
- Root Mean Squared Log Error (**RMSLE**)
- R Squared (**R²**)

Let's go over them and see their advantages and disadvantages.

Evaluation metrics

Mean Absolute Error (MAE):

Calculates the absolute difference between actual and predicted values:

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

Diagram illustrating the MAE formula components:

- $\frac{1}{n}$: Divide by the total number of data points
- $\sum$: Sum of
- y : Actual output value
- $\hat{y}$: Predicted output value
- $|y - \hat{y}|$: The absolute value of the residual

Advantages:

- The MAE you get is in the same unit as the output variable.
- It is most Robust to outliers.

Disadvantages:

- The graph of MAE is not differentiable so we have to apply various optimizers like Gradient descent which can be differentiable.

Evaluation metrics

Mean Squared Error (MSE):

Calculates the squared difference between actual and predicted values. By squaring the difference, we avoid the cancellation of negative terms and it is the benefit of MSE.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Diagram annotations:

- 1 : average over all results
- N : average over all results
- y_i : true y
- $\hat{y}_i$: estimate of y
- 2 : makes result quadratic

Advantages:

- The graph of MSE is differentiable, so you can easily use it as a loss function.

Disadvantages:

- The value you get after calculating MSE is a squared unit of output.
- If you have outliers in the dataset then it penalizes the outliers most and the calculated MSE is bigger. It is not Robust to outliers which were an advantage in MAE.

Evaluation metrics

Root Mean Squared Logarithmic Error (RMSLE):

Calculates the logarithmic square root of MSE, a bias of 1 is added to avoid errors:

$$RMSLE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(1 + \hat{y}_i) - \log(1 + y_i))^2}$$

Advantages:

- RMSLE value will only consider the relative error between **predicted** and the **actual** value neglecting the scale of data.
- RMSLE incurs a larger penalty for the **underestimation** of the actual variable than the **overestimation**.
- Works best with skewed distributions.

Disadvantages:

- The result is in a different unit than the output variable.
- Sensitive to small predictions for large true values: a close to zero prediction for a large actual value leads to a very large log error, which may disproportionately dominate the RMSLE.

Evaluation metrics

Mean Average Percentage Error

Calculates the **average percentage difference** between predicted and actual values:

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100$$

Advantages:

- Easy to interpret — expressed as a percentage error.
- Scale independent — allows comparison across different datasets.
- Intuitive for business and forecasting applications.

Disadvantages:

- Undefined when the true value $y_i=0$.
- Extremely sensitive to small true values, which can produce very large percentage errors.
- Penalizes overestimation and underestimation asymmetrically in some situations.

Evaluation metrics

R Squared (R²)

Measures how well a regression model explains the variance in the target variable. It's a statistical metric used to evaluate the goodness of fit of a model:

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- ❖ $R^2 = 1 \rightarrow$ Perfect prediction; all points lie exactly on the prediction line.
- ❖ $R^2 = 0 \rightarrow$ Model predicts no better than the mean of the data.
- ❖ $R^2 < 0 \rightarrow$ Model is worse than simply predicting the mean.

Advantages:

- Easy to interpret – shows how well the model explains data variability.
- Good for model comparison – higher R^2 usually means a better fit.
- Unitless and scale-independent – allows comparisons across datasets.

Disadvantages:

- Doesn't measure prediction accuracy – high R^2 doesn't mean low error.
- Can be inflated by adding variables – risks overfitting.
- Misleading in nonlinear models or with outliers – can distort results.

Evaluation metrics

Adjusted R²

A limitation of R²:

Adding more variables **always increases R²**, even if the features are irrelevant.

Adjusted R² corrects for this by penalizing model complexity:

$$R_{adj}^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

Where:

n = number of observations

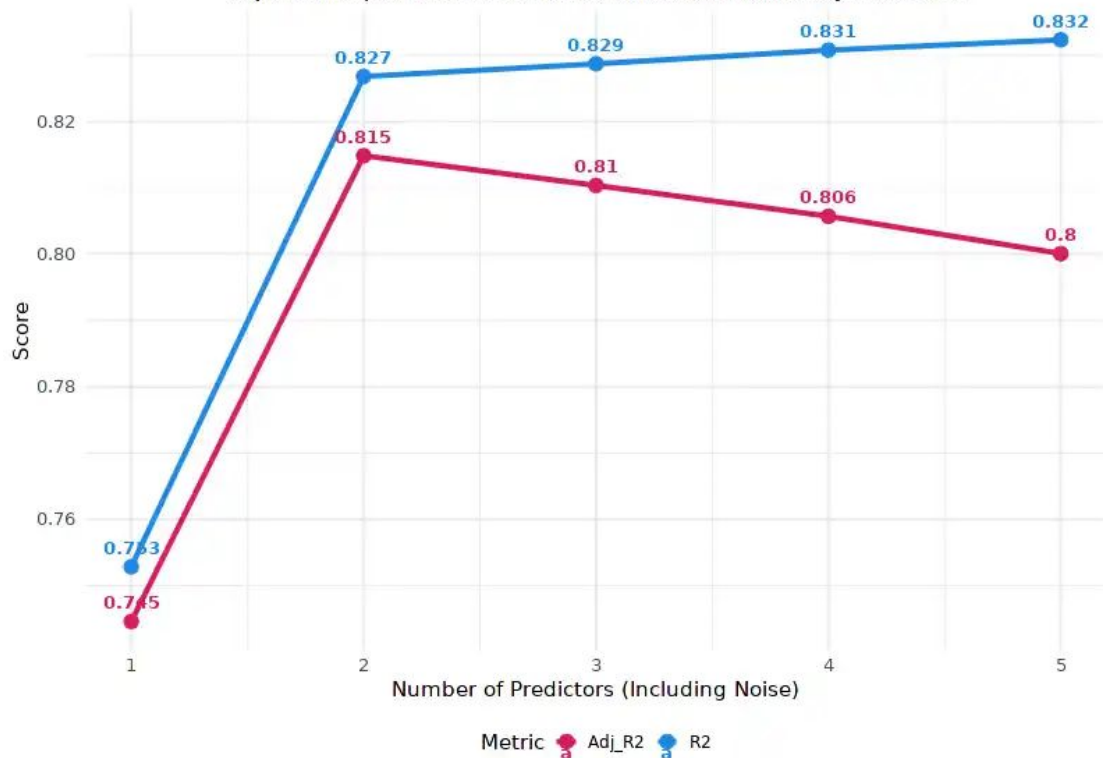
p = number of predictors

Adjusted R² increases **only if new variables improve the model sufficiently**.

R2 vs Adjusted R2

R2 vs. Adjusted R2 Using Real-World Data (mtcars)

Adjusted R2 penalizes added noise, unlike R2 which always increases



Evaluation Metrics

Why do we need evaluation metrics?

Let's say we have trained a few models, that perform well on the train data. How can we compare said models? What sort of metrics can we use to choose the best model out of them, or to see if our model is even viable in the "real world"? For classification models, the most commonly used metrics are:

- Logarithmic Loss (Log Loss)
- Accuracy
- Precision
- Recall
- F1 Score
- Area Under Curve (AUC) and ROC Curve

Let's go over them and see their advantages and disadvantages.

Evaluation Metrics

Logarithmic Loss (Log Loss):

Log loss measures the uncertainty of the model's predictions. It is calculated by penalizing the model for assigning low probabilities to the correct classes. This metric is used in multi-class classification and is helpful when we want to assess a model's confidence in its predictions. If there are N samples belonging to the M class, then we calculate the Log loss in this way:

$$\text{Logarithmic Loss} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \cdot \log(p_{ij})$$

Evaluation Metrics

	Actual positive	Actual negative
Predicted positive	True positive (TP): A spam email correctly classified as a spam email. These are the spam messages automatically sent to the spam folder.	False positive (FP): A not-spam email misclassified as spam. These are the legitimate emails that wind up in the spam folder.
Predicted negative	False negative (FN): A spam email misclassified as not-spam. These are spam emails that aren't caught by the spam filter and make their way into the inbox.	True negative (TN): A not-spam email correctly classified as not-spam. These are the legitimate emails that are sent directly to the inbox.

Evaluation Metrics

Accuracy:

Accuracy is a fundamental metric used for evaluating the performance of a classification model. It tells us the proportion of correct predictions made by the model out of all predictions.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Advantages:

- Simple

Disadvantages:

- Can be misleading in cases of imbalanced datasets.
- Gives a False Positive sense of achieving high accuracy.

Evaluation Metrics

Precision:

Precision measures how many of the positive predictions made by the model are actually correct. It's useful when the cost of false positives is high such as in medical diagnoses where predicting a disease when it's not present can have serious consequences.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Advantages:

- Effective with Imbalanced Data
- Accuracy of Positive Predictions

Disadvantages:

- Ignores False Negatives, which can lead to overly conservative models that miss many actual positive cases.
- Focus on the Positive Class, as the metric doesn't account for the model's ability to find all relevant instances.

Evaluation Metrics

Recall (True Positive Rate):

Recall or Sensitivity measures how many of the actual positive cases were correctly identified by the model. It is important when missing a positive case (false negative) is more costly than false positives.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Advantages:

- In scenarios where catching all positive cases is important (like disease detection), recall is a key metric.

Disadvantages:

- Ignores False Positives, which can lead to overly liberal models that predict everything as positive.

Evaluation Metrics

- Does the patient have cancer? → cost of false negatives is high
Use Recall
- Is this wine good or not? → cost of false positives is high
Use Precision

Evaluation Metrics

F1 Score:

The F1 Score is the harmonic mean of precision and recall. It is useful when we need a balance between precision and recall as it combines both into a single number.

- Lower recall and higher precision gives us great accuracy but then it misses a large number of instances.
- A high F1 score means the model performs well on both metrics. Its range is [0,1].
- More the F1 score better will be performance. It can be expressed mathematically in this way:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Evaluation Metrics

Area Under Curve (AUC) and ROC Curve:

First we need a few additional terms - **Specificity (True Negative Rate)**, which measures how many actual negative instances were correctly identified by the model. It answers the question: "Out of all the actual negative cases, how many did the model correctly identify as negative?"

$$\text{TNR} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

And **FPR (False Positive Rate)** - It measures how many actual negative instances were incorrectly classified as positive.

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

Evaluation Metrics

Area Under Curve (AUC) and ROC Curve:

ROC Curve is the graphical representation of the **True Positive Rate (TPR)** vs the **False Positive Rate (FPR)** at different classification thresholds. The curve helps us visualize the trade-offs between sensitivity (**TPR**) and specificity (**1 - FPR**) across various thresholds.

Area Under Curve (AUC) quantifies the overall ability of the model to distinguish between positive and negative classes.

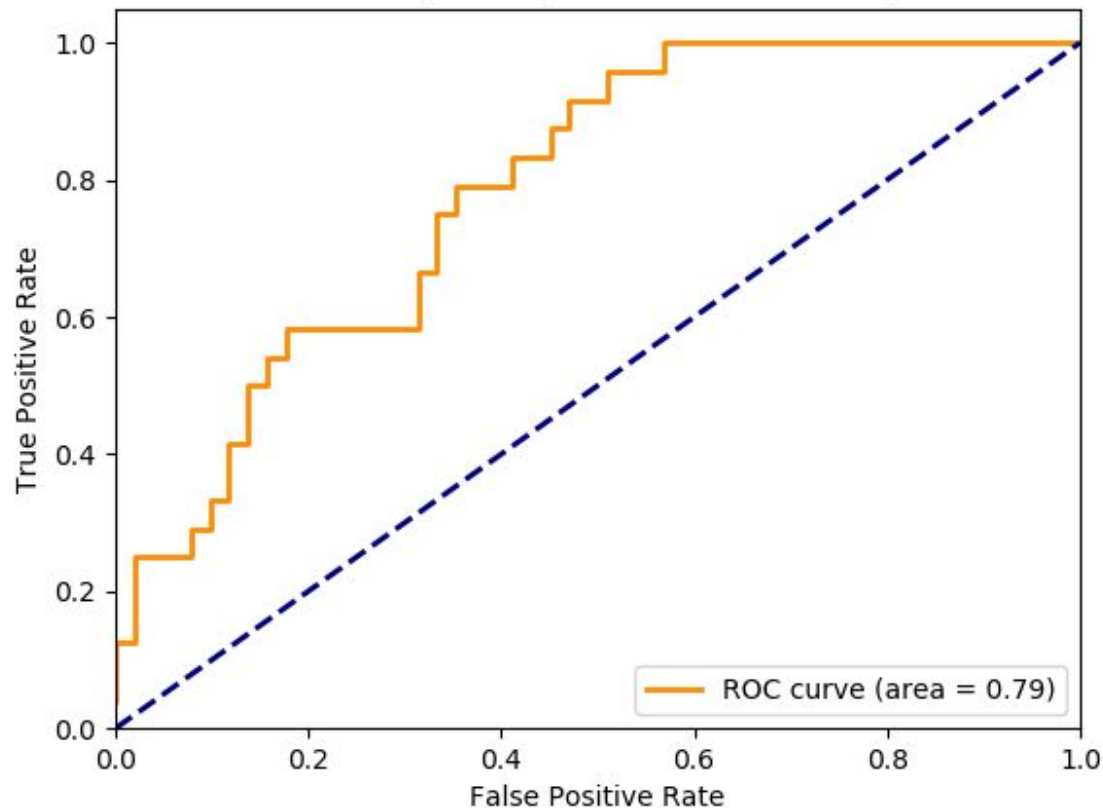
- AUC = 1: Perfect model (always correctly classifies positives and negatives).
- AUC = 0.5: Model performs no better than random guessing.
- AUC < 0.5: Model performs worse than random guessing (showing that the model is inverted).

ROC curves are particularly useful when:

- Datasets are **imbalanced**
- Classification thresholds must be tuned

Evaluation Metrics

Receiver operating characteristic example



Model Evaluation

Training Error vs Test Error

As model complexity increases:

Training error ↓

Test error:

- Decreases initially
- Then increases due to **overfitting**

The optimal model lies at the **minimum test error**

Regularization helps us reach this point.

